



Relational Model, SQL & PLSQL

Mr. S.K.Patil

Relational Model, SQL & PLSQL

Relational Model –

- The relational model is the primary data model for commercial data processing applications.
- It is at primary position because of its simplicity.
- **Structure of Relational Databases –**
 - A relational database consists of a collection of tables, each of which is assigned a unique name.
 - e.g. Student table stores information about students.
 - Each table has columns and rows.
 - A row in a table represents a relationship among a set of values.
 - Since a table is a collection of such relationships, there is a close correspondence between the concept of table and the mathematical concept of relation.

Relational Model, SQL & PLSQL

Relational Model –

➤ Structure of Relational Databases –

- In the relational model the term relation is used to refer to a table, while the term tuple is used to refer to a row.
- The order in which tuples appear in a relation is irrelevant, since a relation is a set of tuples.
- The term attribute refers to a column of a table.
- For each attribute of a relation, there is a set of permitted values, called the domain of that attribute.
- The null value is a special value that signifies that the value is unknown or does not exist.

Relational Model, SQL & PLSQL

Relational Model –

➤ Database Schema –

- The database schema is the logical design of the database. (Schema – variable type definition)
- The database instance is a snapshot of the data in the database at a given instant in time. (instance – value of variable)
- A relation schema consists of a list of attributes and their corresponding domains.
- The contents of a relation instance may change with time as the relation is updated.
- The schema of a relation does not generally change.

Relational Model, SQL & PLSQL

Relational Model –

➤ Keys –

- The values of the attribute of a tuple must uniquely identify the tuple.
- No two tuples in a relation are allowed to have exactly the same value for all attributes.
- A superkey is a set of one or more attributes that, taken collectively, allow us to identify uniquely a tuple in the relation.
- The minimal superkeys are called candidate keys.
- It is possible that several distinct sets of attributes could serve as a candidate key.
- The primary key is a candidate key that is chosen by the database designer to identify tuples within a relation.
- The primary key should be chosen such that its attribute values are never, or very rarely, changed.

Relational Model, SQL & PLSQL

Relational Model –

➤ Keys –

- A relation, say r1, may include the primary key of another relation, say r2, as an attribute. This attribute is called a foreign key from r1, referencing r2.
- The relation r1 is called the referencing relation of the foreign key, and r2 is called the referenced relation of the foreign key.

➤ Schema Diagrams –

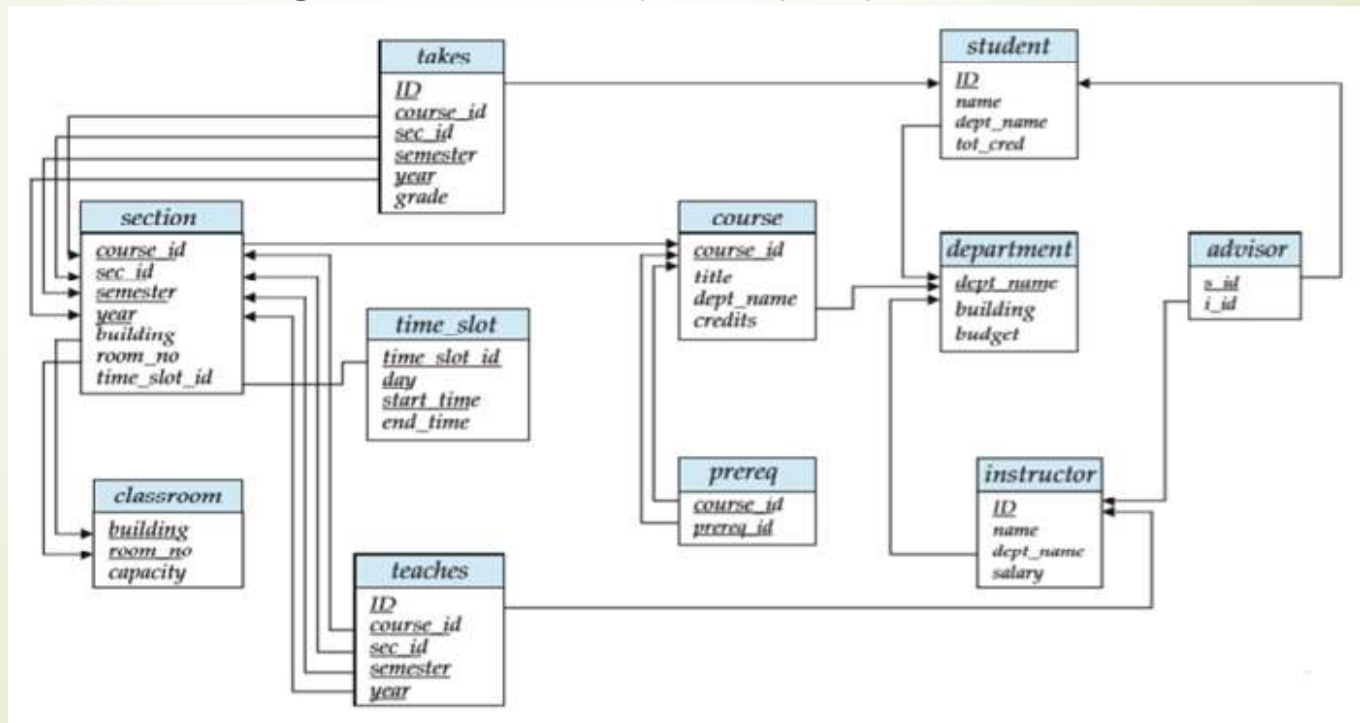
- A database schema, along with primary key and foreign key dependencies, can be shown by schema diagrams.
- Below figure shows the schema diagram for university organization.
- Each relation appears as a box, with the relation name at the top in blue, and the attributes listed inside the box.
- Primary key attributes are shown underlined.

Relational Model, SQL & PLSQL

Relational Model –

➤ Schema Diagrams –

- Foreign key dependencies appear as arrows from the foreign key attributes of the referencing relation to the primary key of the referenced relation.



Relational Model, SQL & PLSQL

Relational Model –

➤ Relational Algebra –

- The relational algebra provides a set of operations that take one or more relations as input and return a relation as an output.
- The query languages such as SQL are based on the relational algebra, but it add a number of useful syntactic features.
- The relational query languages define a set of operations that operate on tables, and output tables as their results.
- These operations can be combined to get expressions that express desired queries.

Relational Model, SQL & PLSQL

Overview of SQL –

- **IBM developed the original version of SQL**, originally called **Sequel**, as **part of the SystemR project** in the **early 1970s**.
- The Sequel language has evolved since then, and its **name** has **changed to SQL (Structured Query Language)**.
- Many products now support the SQL language. **SQL** has clearly established itself as the **standard relational database language**.
- **In 1986**, the American National Standards Institute (**ANSI**) and the International Organization for Standardization (**ISO**) **published an SQL standard**, called **SQL-86**.
- ANSI published an **extended standard for SQL, SQL-89, in 1989**.
- The **next version** of the standard was **SQL-92** standard, followed by **SQL:1999, SQL:2003, SQL:2006, SQL:2008** and **most recently SQL:2023**.

Relational Model, SQL & PLSQL

Overview of SQL –

- The **SQL language** has several parts:
 - **Data-definition language** - The **SQL DDL** provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
 - **Data-manipulation language** - The **SQL DML** provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.
 - **Integrity** - The **SQL DDL** includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.
 - **View definition** - The **SQL DDL** includes commands for defining views.
 - **Transaction control** - **SQL** includes commands for specifying the beginning and ending of transactions.

Relational Model, SQL & PLSQL

Overview of SQL –

- SQL is the database language used to **perform certain operations on the existing database** and **also used to create new database**.
- It **uses commands** to **carry out the required tasks**.
- SQL **commands** are like **instructions to a table**.
- It is **used to interact with the database with some operations**.
- It is also **used to perform specific tasks, functions and queries of data**.
- SQL can perform various tasks like **creating a table, adding data to tables, dropping the table, modifying the table, set permissions for users**.

Relational Model, SQL & PLSQL

Overview of SQL –

- **SQL commands** are mainly **categorized into five categories** –
 - **DDL** (Data Definition Language) **Create, Drop, Alter, Truncate**
 - **DML** (Data Manipulation Language) **Insert, Update, Delete**
 - **DQL** (Data Query Language) **Select**
 - **DCL** (Data Control Language) **Grant, Revoke**
 - **TCL** (Transaction Control Language) **Commit, Rollback, Save point**

Relational Model, SQL & PLSQL

SQL Data Definition –

- The **set of relations in a database** must be **specified to the system using a data-definition language (DDL)**.
- The SQL DDL allows **specification of not only a set of relations**, but also **information about each relation**, including:
 - The **schema for each relation**.
 - The **types of values associated with each attribute**.
 - The **integrity constraints**.
 - The **set of indices** to be maintained for each relation.
 - The **security and authorization information** for each relation.
 - The **physical storage structure** of each relation on disk.

Relational Model, SQL & PLSQL

SQL Data Definition –

- **Data Types** - The SQL standard supports a **variety of built-in types**, including:
- **char(n)**: A **fixed-length character string** with user-specified **length n**. The full form, **character**, can be used instead.
- **varchar(n)**: A **variable-length character string** with user-specified maximum **length n**. The full form, **character varying**, is equivalent.
- **int**: An integer (a finite subset of the integers that is machine dependent). The full form, **integer**, is equivalent.
- **smallint**: A small integer (a machine-dependent subset of the integer type).
- **numeric(p,d)**: A **fixed-point number with user-specified precision**. The number consists of **p digits** (plus a sign), and **d of the p digits are to the right of the decimal point**. e.g. **numeric(3,1)** allows 44.5 to be stored exactly, but neither 444.5 or 0.32 can be stored exactly in a field of this type.

Relational Model, SQL & PLSQL

SQL Data Definition –

- **real, double precision:** Floating-point and double-precision floating-point numbers with machine-dependent precision.
- **float(n):** A floating-point number, with precision of at least n digits.
- **date:** A calendar date containing a (four-digit) year, month, and day of the month.
- **time:** The time of day, in hours, minutes, and seconds. A variant, **time(p)**, can be used to specify the number of fractional digits for seconds (the default being 0). It is also possible to store time-zone information along with the time by specifying time with timezone.
- **timestamp:** A combination of date and time. A variant, **timestamp(p)**, can be used to specify the number of fractional digits for seconds (the default here being 6). Time-zone information is also stored if with timezone is specified.

Relational Model, SQL & PLSQL

SQL Data Definition –

- Each type may include a **special value** called the **null value**.
- A **null value** indicates an **absent value** that **may exist** but be **unknown** or that **may not exist at all**.
- The **char data type stores fixed length strings**. e.g. attribute A of type char(10). If we store a string “Abc” in this attribute, 7 spaces are appended to the string to make it 10 characters long.
- In contrast, if attribute B were of type varchar(10), and we store “Abc” in attribute B, no spaces would be added.
- **Always use the varchar type** instead of the char type.
- SQL also provides the **nvarchar type** to **store multilingual data** using the Unicode representation.
- However, many databases allow Unicode (in the UTF-8 representation) to be stored even in varchar types.

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

- **DDL** actually **consists of** the **SQL commands** that can be **used to define the database schema**.
- It simply **deals with descriptions of the database schema**.
- It is **used to create** and **modify the structure of database objects** in the database.
- DDL is a **set of SQL commands used to create, modify, and delete database structures but not data**.
- These commands are **normally not used by a general user**, who should be accessing the database via application.
- All the command of DDL are auto-committed that means it permanently save all the changes in the database.

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ Create -

- In SQL, **Create table** command is **used to create a table in database**.
- A **table** is a combination of **rows and columns**.
- For creating a table, we have to define the structure of a table by adding names to columns and providing datatype and size of data to be stored in columns.

➤ Syntax –

```
CREATE TABLE table_name  
( column1 datatype(size),  
  column2 datatype(size), .....  
  column datatype(size)  
);
```

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ Create -

- The **general form** of the create table command is:

```
create table r
  (A1 D1,
   A2 D2,
   ...,
   An Dn,
   (integrity-constraint1),
   ...,
   (integrity-constraintk));
```

Where, r is the name of relation,

each A_i is the name of an attribute

D_i is the domain of attribute A_i which specifies type of attribute A_i

integrity constraint is a rule that each data must follow

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

- Create -

- Example –

```
create table department
(dept_name varchar (20),
building   varchar (15),
budget     numeric (12,2),
primary key (dept_name));
```

- The above relation **department** has three attributes –
 - **dept_name** is a character string of maximum length 20
 - **building** is a character string of maximum length 15
 - **budget** is a number with 12 digits in total, 2 of which are after the decimal point.
 - The create table command also specifies that the **dept_name** attribute is the **primary key** of the department relation.

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ SQL Constraints -

- **Used to specify rules** for the data in a table.
- Constraints are **used to limit the type of data** that can go into a table.
- This **ensures** the **accuracy and reliability of the data** in the table.
- If there is **any violation between** the **constraint and** the **data action**, the **action is aborted**.
- **Constraints** can be **column level** or **table level**.
- **Column level** constraints **apply to a column**, and **Table level** constraints **apply to the whole table**.

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ SQL Constraints -

- **NOT NULL** - Ensures that a **column cannot have a NULL value**.
- **UNIQUE** - Ensures that **all values in a column are different**.
- **PRIMARY KEY**- A combination of a NOT NULL and UNIQUE. **Uniquely identifies each row in a table**.
- **FOREIGN KEY** - **Uniquely identifies a row/record in another table**
- **CHECK** - Ensures that **all values in a column satisfies a specific condition**
- **DEFAULT** - **Sets a default value** for a column when no value is specified

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ SQL Constraints – NOT NULL Constraint

- If a **column** in a table is **specified as Not Null**, then it's **not possible to insert a null in such column**.
- It can be **implemented with create and alter commands**.
- When we implement the Not Null constraint with alter command there should not be any null values in the existing table.

➤ Example –

```
CREATE TABLE Persons  
( Person_id int NOT NULL,  
  Name varchar(50) NOT NULL,  
  Address varchar(100),  
  PRIMARY KEY (Person_id) );
```

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ SQL Constraints – UNIQUE Constraint

- The **unique constraint** doesn't allow **duplicate values** in a column.
- If unique constraint encompasses two or more columns, no two equal combinations are allowed.

➤ Example –

```
CREATE TABLE Persons
```

```
( Person_id int NOT NULL UNIQUE,
```

```
  Name varchar(50) NOT NULL,
```

```
  Address varchar(100));
```


Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ SQL Constraints – PRIMARY KEY Constraint

- The **PRIMARY KEY** constraint **uniquely identifies each record** in a table.
- Primary keys **must contain UNIQUE values**, and **cannot contain NULL values**.
- A table can have only **ONE primary key**; and in the table, this primary key can consist of single or multiple columns (fields).

➤ Example –

```
CREATE TABLE books  
( book_id NUMBER PRIMARY KEY,  
  book_name VARCHAR2(30),  
  author_name VARCHAR2(40),  
  book_isbn VARCHAR2(20) );
```

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

- SQL Constraints – FOREIGN KEY Constraint
 - A **FOREIGN KEY** is a key **used to link two tables together**.
 - A FOREIGN KEY is a **field** (or collection of fields) **in one table** that **refers to the PRIMARY KEY in another table**.
 - The **table containing the foreign key** is called the **child table**, and the **table containing the candidate key** is called the **referenced or parent table**.

```
create table department
(dept_name  varchar (20),
building   varchar (15),
budget     numeric (12,2),
primary key (dept_name));
```

```
create table course
(course_id  varchar (7),
title      varchar (50),
dept_name  varchar (20),
credits    numeric (2,0),
primary key (course_id),
foreign key (dept_name) references department);
```

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ SQL Constraints – CHECK Constraint

- **CHECK constraint** is used to **limit the value range** that can be **placed in a column**.
- If you define a **CHECK constraint on a single column** it **allows only certain values for this column**.
- If you define a **CHECK constraint on a table** it can **limit the values in certain columns based on values in other columns in the row**.

➤ Example –

```
CREATE TABLE Persons (  
    P_ID int NOT NULL CHECK(P_ID>0),  
    Name varchar(50) NOT NULL  
    City varchar (25) )
```

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ SQL Constraints – DEFAULT Constraint

- The **DEFAULT constraint** is **used to insert a default value** into a column.
- The **default value** will be **added to all new records**, if **no other value is specified**.

➤ Example –

```
CREATE TABLE Persons  
( Person_id int NOT NULL,  
  Name varchar(50) NOT NULL,  
  Address varchar(100),  
  City varchar(50) DEFAULT 'Kolhapur');
```

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ Alter -

- The SQL **ALTER TABLE** command is used to **add, delete or modify columns** in an **existing table**.
- It is also used to **add and drop various constraints** on an existing table.

➤ Syntax -

- **Alter Table to add a new column in an existing table -**

ALTER TABLE table_name ADD column_name datatype;

- **Drop Column from existing table –**

ALTER TABLE table_name DROP COLUMN column_name;

- **Alter Table to change the DATATYPE of a column in a table –**

ALTER TABLE table_name MODIFY column_name datatype;

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ Alter -

➤ Syntax -

➤ Alter Table to add a NOT NULL constraint to a column -

```
ALTER TABLE table_name MODIFY column_name datatype NOT NULL;
```

➤ Alter Table to ADD UNIQUE CONSTRAINT to a table -

```
ALTER TABLE table_name ADD CONSTRAINT MyUniqueConstraint  
UNIQUE(column1, column2...);
```

➤ Alter Table to ADD CHECK CONSTRAINT to a table -

```
ALTER TABLE table_name ADD CONSTRAINT MyUniqueConstraint CHECK  
(CONDITION);
```

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ Alter -

➤ Syntax -

➤ **Alter Table to ADD PRIMARY KEY constraint to a table -**

```
ALTER TABLE table_name ADD CONSTRAINT MyPrimaryKey PRIMARY KEY  
(column1, column2...);
```

➤ **Alter Table to DROP CONSTRAINT from a table -**

```
ALTER TABLE table_name DROP CONSTRAINT MyUniqueConstraint;
```

➤ **Alter Table to DROP PRIMARY KEY constraint from a table -**

```
ALTER TABLE table_name DROP CONSTRAINT MyPrimaryKey;
```

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ Drop -

- **DROP** is used to **delete a whole database** or just a table.
- DROP statement **destroys objects** like an existing database, table, index, or a view.
- A DROP statement in SQL **removes a component from RDBMS.**

➤ Syntax –

DROP object object_name;

➤ Drop a table-

DROP TABLE table_name;

➤ Drop a database-

DROP DATABASE database_name;

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ Truncate -

- **TRUNCATE** is used to **delete the data inside the table not the whole table**.
- The result of this operation quickly removes all data from a table, typically bypassing integrity-enforcing mechanisms.

➤ Syntax –

TRUNCATE TABLE table_name;

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ DROP vs TRUNCATE –

DROP	TRUNCATE
In this, data and its definition is deleted with their full structure.	It preserves the structure of the table for further use but deletes all data.
It is used to eliminate existing complications in the whole database from the table.	It is used to eliminate the tuples from the table.
Integrity constraints get removed .	Integrity constraints doesn't get removed .
View of the table does not exist as structure does not exist	View of the table exist as structure exist.
Free the table space from memory	Does not free the table space from memory
Slow as compared to TRUNCATE because of so many complications	Fast as compared to DROP because of fewer complications

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ Rename -

- SQL provides **two ways to rename the table**. i.e. either RENAME or ALTER.

➤ Syntax –

RENAME old_object_name TO new_object_name;

- **e.g.** RENAME TABLE Customers to Buyers

Relational Model, SQL & PLSQL

Data Definition Language (DDL) –

➤ Sample Queries -

- Create a table customer with customer_name, customer_street, customer_city with primary key customer_name.
- Create a table account with account_number, branch_name, balance with restriction balance value should greater than equal to 0 and primary is account_number.
- Create a table branch with name, city and asset with restriction as city should not null, asset value should greater than equal to 0 and default value is 0 and primary key name.

Relational Model, SQL & PLSQL

Data Manipulation Language (DML) –

- The **DML** commands in SQL **change the data** present in the SQL database.
- Easily **store, modify, update and delete** the existing records from the database using DML commands.
- The command of DML is not auto-committed that means it can't permanently save all the changes in the database. They can be rollback.
- Following are the DML commands –
 - **INSERT** – to add rows to a table
 - **UPDATE** – to change column values of existing rows
 - **DELETE** – to remove rows from a table.

Relational Model, SQL & PLSQL

Data Manipulation Language (DML) –

➤ INSERT -

- Most important DML command in SQL which allows users to **insert data in database tables**.
- It is a request **to insert one tuple**.
- Following points should be considered while inserting data -
 - SQL uses all the columns by default if you do not specify the column name while inserting a row.
 - The number of columns in the list of column name must match the number of values that appear in parenthesis after the word “values”.
 - The datatype for a column and its corresponding value must match.

Relational Model, SQL & PLSQL

Data Manipulation Language (DML) –

➤ INSERT -

➤ Syntax –

insert into table_name (col1, col2, ... coln) values (val1, val2,...valn);

➤ Insert into for all attributes –

➤ **insert into table_name values (val1, val2,...valn);**

➤ e.g. insert into Student values ('EN1234', 'abc', 'DS', 123456)

➤ Insert for some attributes –

➤ **insert into table_name (col_names) values (values to be inserted in resp col);**

➤ e.g. insert into employees (employee_id, last_name, email, hire_date, job_id, salary) values (207, 'Gregory', 'pgregory@oracle.com', sysdate, 'CLERK', 50000);

Relational Model, SQL & PLSQL

Data Manipulation Language (DML) –

➤ INSERT -

➤ Use of select in insert –

- e.g. insert into bonuses SELECT employee_id, salary*1.1 FROM employees WHERE last_name="Gregory";

➤ Sample Queries –

- Insert an account A-978245 at the Park Street branch and that is has a balance of Rs. 12000.
- Insert an customer Jacob who live in street is main street and city is London

Relational Model, SQL & PLSQL

Data Manipulation Language (DML) –

➤ UPDATE -

- It is used to **update existing data within a table**.
- We can **update single columns as well as multiple columns** using the UPDATE statement as per requirement.

➤ Syntax –

UPDATE table_name

SET attr1=val1, attr2=val2,....

[WHERE condition];

- WHERE – Specifies the condition which we want to implement on the table
- SET – Used to set new values to the particular column.

Relational Model, SQL & PLSQL

Data Manipulation Language (DML) –

➤ UPDATE -

- Following points should be remembered while executing Update statement-
 - It references only one table
 - In the SET clause at least one column must be assigned an expression for the update statement
 - In the where clause you could also give multiple conditions for update statement.
- Example –
 - `UPDATE employees SET salary = salary + 1000.0;`

Relational Model, SQL & PLSQL

Data Manipulation Language (DML) –

➤ UPDATE -

➤ Sample Queries –

- Change the assets of Perryridge branch to 340000000.
- Add 2% interest to all bank account balances with a balance of \$500 or less.
- An annual interest payments are being made, interest of 5% to be paid only to accounts with a balance of \$1000 or more

Relational Model, SQL & PLSQL

Data Manipulation Language (DML) –

➤ DELETE -

- It is used to **delete records from a database table**.
- It can be used to **selectively remove records** from a database table **based on certain conditions**.
- It is important for **database size management, data accuracy** and **integrity**.

➤ Syntax –

DELETE FROM table_name

WHERE condition;

- DELETE is a DML command hence operation performed by it can be rolled back or undone.

Relational Model, SQL & PLSQL

Data Manipulation Language (DML) –

➤ DELETE -

- We can delete single as well as multiple records depending on the condition we provide in the WHERE clause.
- If we omit the WHERE clause then all of the records will be deleted and the table will be empty.

➤ Examples –

- DELETE FROM suppliers WHERE supplier_name = 'IBM';
- DELETE FROM suppliers;

➤ Sample Queries –

- Delete the branch Perryridge.
- Remove all the customer who live in “Downtown”

Relational Model, SQL & PLSQL

Data Query Language (DQL) –

- DQL statements are used for performing queries on the data within schema objects.
- The purpose of the DQL command is to get some schema relation based on the query passed to it.
- It is component of SQL statement that allows getting data from the database and imposing order on it.
- It includes the SELECT statement.
- This command allows getting the data out of the database to perform operations with it.

Relational Model, SQL & PLSQL

Data Query Language (DQL) –

➤ SELECT Statement –

- The SELECT statement in SQL is used to fetch or retrieve data from a database.
- It allows users to access the data and retrieve specific data based on specific conditions.
- We can fetch either entire table or according to some specified rules.
- The data returned is stored in a result table. The result table is also called as result set.
- Retrieving/ Fetching Data from a table –
 - SQL queries use the SELECT statement
 - The basic structure of an SQL expression consists of three clauses: select, from, and where.

Relational Model, SQL & PLSQL

Data Query Language (DQL) –

➤ SELECT Statement –

- Retrieving/ Fetching Data from a table –
 - The select clause corresponds to which column do we want to see in result.
 - The from clause corresponds to which table(s) is (are) needed for retrieval.
 - The where clause corresponds to what condition to filter the rows.
- General form –

SELECT A1, A2, ...An

FROM r1, r2, ...

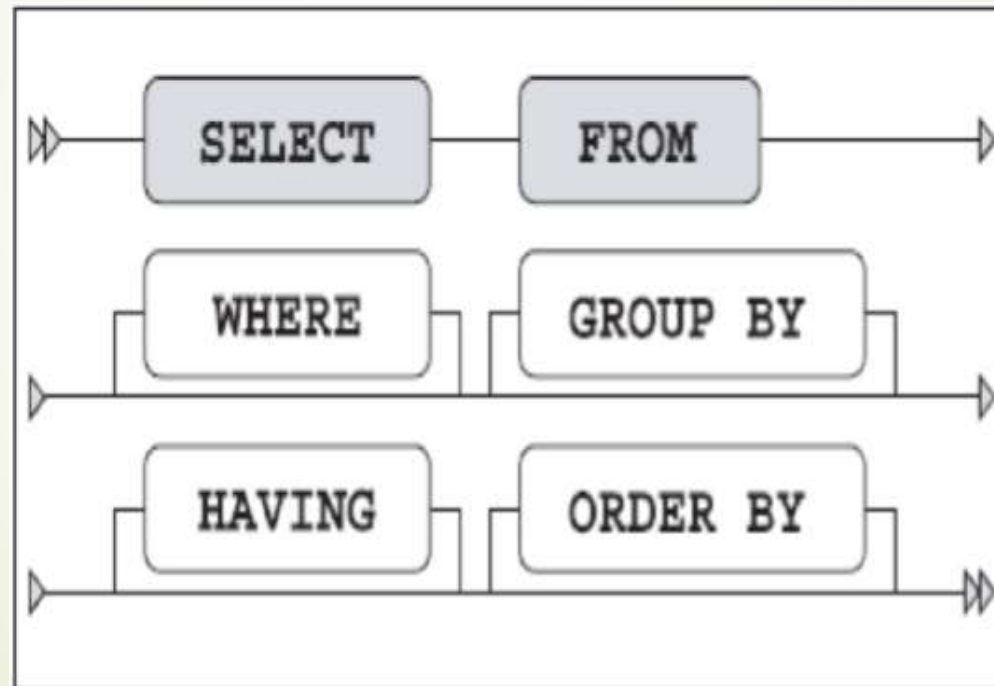
WHERE P;

Relational Model, SQL & PLSQL

Data Query Language (DQL) –

➤ SELECT Statement –

- Six main components of SELECT command –



Relational Model, SQL & PLSQL

Data Query Language (DQL) –

➤ SELECT Statement –

- Six main components of SELECT command –

Component	Description
FROM	Which table(s) is (are) needed for retrieval?
WHERE	What is the condition to filter the rows?
GROUP BY	How should the rows be grouped/aggregated?
HAVING	What is the condition to filter the aggregated groups?
SELECT	Which columns do you want to see in the result?
ORDER BY	In which order do you want to see the resulting rows?

Relational Model, SQL & PLSQL

Data Query Language (DQL) –

➤ SELECT Statement –

➤ Examples -

➤ List all the employee Details –

➤ Select* from employees;

➤ SELECT FIRST_NAME, LAST_NAME, DEPARTMENT_ID FROM EMPLOYEES;

➤ Find the names of all branches in the loan relation

➤ SELECT all branch_name FROM loan;

➤ SELECT branch_name FROM loan ;

➤ Get all distinct branches in the loan relation –

➤ SELECT Distinct branch_name FROM loan;

Relational Model, SQL & PLSQL

Data Query Language (DQL) –

➤ SELECT Statement –

➤ Examples -

➤ Selecting Data that Satisfies Specified Conditions –

➤ `SELECT FIRST_NAME, LAST_NAME, DEPARTMENT_ID FROM EMPLOYEES
WHERE DEPARTMENT_ID = 90;`

➤ Find all loan numbers for loans with loan amounts greater than \$1200 –

➤ `select loan-no from loan where amount > 1200`

Relational Model, SQL & PLSQL

Data Control Language (DCL) –

- DCL includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions, and other controls of the database system.
- DCL helps users to retrieve and modify the data stored in the database with some specified queries.
- Grant and Revoke belong to these types of commands of the Data controlling Language.
- DCL is a component of SQL Commands.
- **Grant –**
 - SQL Grant command is specifically used to provide privileges to database objects for a user.
 - This command also allows users to grant permissions to other users too.

Relational Model, SQL & PLSQL

Data Control Language (DCL) –

➤ Grant –

➤ Syntax –

```
grant privilege_name on object_name  
to {user_name | public | role_name}
```

- privilege_name is which permission has to be granted
- object_name is the name of the database object
- user_name is the user to which access should be provided
- public is used to permit access to all the users

Relational Model, SQL & PLSQL

Data Control Language (DCL) –

➤ Revoke –

- Revoke command withdraw user privileges on database objects if any granted.
- It does operations opposite to the Grant command.
- When a privilege is revoked from a particular user U, then the privileges granted to all other users by user U will be revoked.
- Syntax –

```
revoke privilege_name on object_name  
from {user_name | public | role_name}
```

Relational Model, SQL & PLSQL

Data Control Language (DCL) –

➤ Example –

```
grant insert,  
select on accounts to Ram
```

- By the above command, user ram has granted permissions on accounts database object like he can query or insert into accounts.

```
revoke insert,  
select on accounts from Ram
```

- By the above command, user ram's permissions like query or insert on accounts database object has been removed.

Relational Model, SQL & PLSQL

Data Control Language (DCL) –

➤ Grant vs. Revoke –

Grant	Revoke
This DCL command grants permissions to the user on the database objects.	This DCL command removes permissions if any granted to the users on database objects.
It assigns access rights to users.	It revokes the user access rights of users.
For each user you need to specify the permissions.	If access for one user is removed; all the particular permissions provided by that users to others will be removed.
When the access is decentralized then granting permissions will be easy.	If decentralized access then removing the granted permissions is difficult.

Relational Model, SQL & PLSQL

Transaction Control Language (TCL) –

- Transactions group a set of tasks into a single execution unit.
- Each transaction begins with a specific task and ends when all the tasks in the group are successfully completed.
- If any of the tasks fail, the transaction fails.
- Therefore, a transaction has only two results: **success** or **failure**.
- List of TCL Commands –
 - BEGIN TRANSACTION
 - COMMIT
 - ROLLBACK
 - SAVEPOINT

Relational Model, SQL & PLSQL

Transaction Control Language (TCL) –

- TCL commands cannot be used while creating tables and are only used with the DML Commands such as INSERT, UPDATE & DELETE.
- **BEGIN TRANSACTION –**
 - It indicates the start point of an explicit or local transaction.
 - Syntax –

```
BEGIN TRANSACTION transaction_name;
```
- **COMMIT –**
 - If everything is in order with all statements within a single transaction, all changes are recorded together in the database is called **committed**.
 - The COMMIT command saves all the transactions to the database since the last COMMIT or ROLLBACK command.
 - Syntax -

```
COMMIT;
```

Relational Model, SQL & PLSQL

Transaction Control Language (TCL) –

➤ COMMIT –

➤ Example –

Student				
Rol_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
3	Sujit	Rohtak	9156253131	20
4	Suresh	Delhi	9156768971	18
3	Sujit	Rohtak	9156253131	20
2	Ramesh	Gurgaon	9652431543	18

- Following query delete those records from the table which have age = 20 and then COMMIT the changes in the database.

```
DELETE FROM Student WHERE AGE = 20;  
COMMIT;
```

Relational Model, SQL & PLSQL

Transaction Control Language (TCL) –

➤ COMMIT –

➤ Example –

➤ Thus, two rows from the table would be deleted and the SELECT statement would look like,



Rol_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
4	Suresh	Delhi	9156768971	18
2	Ramesh	Gurgaon	9652431543	18

Relational Model, SQL & PLSQL

Transaction Control Language (TCL) –

➤ ROLLBACK –

- If any error occurs with any of the SQL grouped statements, all changes need to be aborted.
- The process of reversing changes is called **rollback**.
- This command can only be used to undo transactions since the last COMMIT or ROLLBACK command was issued.
- Syntax - ROLLBACK;

Relational Model, SQL & PLSQL

Transaction Control Language (TCL) –

➤ **SAVEPOINT –**

- It creates points within the groups of transactions in which to ROLLBACK.
- A SAVEPOINT is a point in a transaction in which you can roll the transaction back to a certain point without rolling back the entire transaction.

➤ Syntax –

SAVEPOINT SAVEPOINT_NAME;

- This command is used only in the creation of SAVEPOINT among all the transactions.
- In general ROLLBACK is used to undo a group of transactions.
- Syntax for rolling back to Savepoint Command:

ROLLBACK TO SAVEPOINT_NAME;

Relational Model, SQL & PLSQL

Transaction Control Language (TCL) –

➤ SAVEPOINT –

- We can ROLLBACK to any SAVEPOINT at any time to return the appropriate data to its original state.
- Example –
- In previous example, Delete those records from the table which have age = 20 and then ROLLBACK the changes in the database by keeping Savepoints.

```
SAVEPOINT SP1;  
//Savepoint created.  
DELETE FROM Student WHERE AGE = 20;  
//deleted  
SAVEPOINT SP2;  
//Savepoint created.
```

- Here SP1 is first SAVEPOINT created before deletion. In this example one deletion have taken place. After deletion again SAVEPOINT SP2 is created.

Relational Model, SQL & PLSQL

Transaction Control Language (TCL) –

➤ SAVEPOINT –

- Example –
- Deletion have been taken place, assume that we have changed our mind & decided to ROLLBACK to the SAVEPOINT SP1 which is before deletion.

```
ROLLBACK TO SP1;  
//Rollback completed
```

- RELEASE SAVEPOINT command is used to remove a SAVEPOINT that we have created.
- Syntax - `RELEASE SAVEPOINT SAVEPOINT_NAME`
- Once a SAVEPOINT has been released, we can no longer use the ROLLBACK command to undo transactions performed since the last SAVEPOINT.

Relational Model, SQL & PLSQL

Additional Basic Operations –

- There are number of additional basic operations that are supported in SQL.

➤ **Rename Operation –**

- SQL provides mechanisms to rename both attributes and relations.
- It uses the “as” clause.
- Syntax - Old-name as new-name
- The ‘as’ clause can appear in both the select and from clause.
- Example – select first_name as name, address, mob_no from student

➤ **String Operations –**

- SQL specifies strings by enclosing them in single quotes, for example, 'Computer'.
- A single quote character that is part of a string can be specified by using two single quote characters; for example string : “It’s right” can be specified by “It”s right”.

Relational Model, SQL & PLSQL

Additional Basic Operations –

➤ String Operations –

- Common operations – equality operation (=), concatenate, extract substrings, finding length of string, convert to uppercase, convert to lowercase, removing space (trim).
- The most commonly used operation on string is pattern matching using the operator **like**.
- We describe patterns by using two special characters –
 - Percent (%): The % character matches any substrings.
 - Underscore: (_): The _ character matches any character.
- Example -
 - 'Intro%' matches any string beginning with "Intro".
 - '%Comp%' matches any string containing "Comp" as a substring

Relational Model, SQL & PLSQL

Additional Basic Operations –

➤ String Operations –

➤ Example -

- '___' matches any string of exactly three characters.
- '___%' matches any string of at least three characters

```
select dept_name  
from department  
where building like '%Watson%';
```

Relational Model, SQL & PLSQL

Additional Basic Operations –

➤ Attribute Specification in Select Clause –

- The asterisk symbol “*” can be used in the select clause to denote “all attributes.”
- A select clause of the form select * indicates that all attributes of the result relation of the from clause are selected.

➤ Ordering the Display of Tuples –

- SQL offers the user some control over the order in which tuples in a relation are displayed.
- The order by clause causes the tuples in the result of a query to appear in sorted order.
- By default, the order by clause lists items in ascending order.
- To specify the sort order, we may specify desc for descending order or asc for ascending order.
- The ordering can be performed on multiple attributes.

Relational Model, SQL & PLSQL

Additional Basic Operations –

➤ Ordering the Display of Tuples –

- Examples –
- List in alphabetic order all instructors in the Physics department

```
select name  
from instructor  
where dept_name = 'Physics'  
order by name;
```

- List the entire instructor relation in descending order of salary. If several instructors have the same salary, we order them in ascending order by name

```
select *  
from instructor  
order by salary desc, name asc;
```

Relational Model, SQL & PLSQL

Additional Basic Operations –

➤ Group by Clause –

- Group by clause is used to group the rows based on certain criteria.
- Group by is usually used in conjunction with aggregate functions like sum, avg, min, max, etc.

- Example - group the employees by the department

```
select emp_name, salary, mob_no, department  
from employee  
group by department
```

Relational Model, SQL & PLSQL

Additional Basic Operations –

➤ Having Clause –

- The having clause tells SQL to include only certain groups produced by the group by clause in the query result set.
- Having is equivalent to the where clause and is used to specify the search criteria or search condition when group by clause is specified.
- Example - Display royalty amount received by authors whose second character name contains 'a'.

```
select author_name, sum (unit_price * .15) as Royalty Amount  
from book  
group by author_name  
having author_name like '_a%'
```


Relational Model, SQL & PLSQL

Additional Basic Operations –

➤ Where Clause –

- The where clause is used to select specific rows satisfying given predicate.
- SQL uses the logical connective and, or and not in the where clause.
- The operands of logical connectives can be expressions involving the comparison operators <, <=, >, >=, =, and <>.
- SQL allows us to use the comparison operators to compare strings and arithmetic expressions, as well as special types, such as date types.
- Example - Find the titles of books published in year 2004

select title

from book

where pub_year = '2004';

Relational Model, SQL & PLSQL

Additional Basic Operations –

➤ Where Clause –

➤ Between –

- SQL includes a between comparison operator to specify that a value be less than or equal to some value and greater than or equal to some other value.
- Example - Find the titles of book having price between 300 to 400

```
select title from book  
where price between 300 and 400;
```

OR

```
select title from book  
where price >= 300 and price <=400;
```

Relational Model, SQL & PLSQL

SET Operations –

- The SQL **Set operation** is used to **combine the two or more SQL SELECT statements**.
- **Types of Set Operation –**
 - Union
 - UnionAll
 - Intersect
 - Minus
- **Union –**
 - It is used to **combine the result of two or more SQL SELECT queries**.
 - In this, all the number of **datatype and columns must be same in both** the tables on which UNION operation is being applied.
 - It **eliminates** the **duplicate rows** from its resultset.

Relational Model, SQL & PLSQL

SET Operations –

➤ Union –

➤ Syntax –

```
SELECT column_name FROM table1
```

```
UNION
```

```
SELECT column_name FROM table2
```

➤ **Example** – Table 1 (id 1,2,3 name- a,b,c) Table 2 (id 3,4,5 name- c,d,e)

➤ UnionAll –

➤ It is **same as UNION** operation. Only **difference** is that it **returns the set without removing duplication and sorting the data.**

➤ **Example** – Table 1 (id 1,2,3 name- a,b,c) Table 2 (id 3,4,5 name- c,d,e)

Relational Model, SQL & PLSQL

SET Operations –

➤ Intersect –

- It is used to **combine two SELECT statements**.
- It **returns the common rows from both the SELECT statements**.
- In this operation, the **number of datatype and columns must be the same**.
- It has **no duplicates** and it **arranges the data in ascending order** by default.

➤ Syntax –

```
SELECT column_name FROM table1
```

```
INTERSECT
```

```
SELECT column_name FROM table2
```

- **Example** – Table 1 (id 1,2,3 name- a,b,c) Table 2 (id 3,4,5 name- c,d,e)

Relational Model, SQL & PLSQL

SET Operations –

➤ Minus –

- It **combines the result of two SELECT statements.**
- It is used to **display the rows** which are **present in the first query** but **absent in the second query.**
- It has **no duplicates** and **data arranged in ascending order by default.**

➤ Syntax –

SELECT column_name FROM table1

MINUS

SELECT column_name FROM table2

- **Example** – Table 1 (id 1,2,3 name- a,b,c) Table 2 (id 3,4,5 name- c,d,e)

Relational Model, SQL & PLSQL

Aggregate Functions –

- Aggregate functions are functions that take a collection (a set or multiset) of values as input and return a single value.
- It is used to summarize data, by combining multiple values to form a single result.
- SQL aggregate functions are mostly used with GROUP BY clause of SELECT statement.
- SQL offers five built-in aggregate functions:
 - Average : avg
 - Minimum : min
 - Maximum : max
 - Total : sum
 - Count : count

Relational Model, SQL & PLSQL

Aggregate Functions –

- The input to sum and avg must be a collection of numbers.
- The other operators can operate on collections of nonnumeric data types, such as strings, as well.

➤ **Average (avg) –**

- The avg function is used to calculate the average value of the numeric type.
- It returns the average of all non-null values.
- Syntax - avg() or avg([All | Distinct] expression)
- Example –

```
select avg (price)
from product;
```


Relational Model, SQL & PLSQL

Aggregate Functions –

➤ Minimum (min) –

- The min function is used to find the minimum value of a certain column.
- The function determines the smallest value of all selected values of a column.
- Syntax - min() or min([All | Distinct] expression)
- Example –

```
select min (price)  
from product;
```

Relational Model, SQL & PLSQL

Aggregate Functions –

➤ **Maximum (max) –**

- The min function is used to find the maximum value of a certain column.
- The function determines the largest value of all selected values of a column.
- Syntax - max() or max([All | Distinct] expression)
- Example –

```
select max (price)  
from product;
```

Relational Model, SQL & PLSQL

Aggregate Functions –

➤ Total (sum) –

- The sum function is used to calculate the sum of all selected columns.
- It works on numeric fields only.
- Syntax - `sum()` or `sum([All | Distinct] expression)`

➤ Example –

```
select sum (price)
from product;
```

➤ Sum with where

```
select sum (price)
from product
where quantity > 5;
```

Relational Model, SQL & PLSQL

Aggregate Functions –

➤ Total (sum) –

➤ Sum with group by

```
select sum (price)
from product
where quantity > 5
group by company;
```

➤ Sum with having

```
select company, sum (price)
from product
group by company
having sum(price) >=500 ;
```

Relational Model, SQL & PLSQL

Aggregate Functions –

➤ Count (count) –

- It is used to count the number of rows in a database table.
- It can work on both numeric and non-numeric datatypes.
- Count function uses the count(*) that returns the count of all rows in a specified table. It considers duplicate and Null.
- Syntax – count (*) or count ([All | Distinct] expression)
- Example –

```
select count (*) from product;
```

- Count with where clause -

```
select count (*) from product  
where price > 50;
```

Relational Model, SQL & PLSQL

Aggregate Functions –

➤ Count (count) –

➤ Count with distinct -

```
select count (distinct company)
from product;
```

➤ Count with group by -

```
select company, count (distinct) from product
group by company;
```

➤ Count with having -

```
select company, count (*) from product
group by company
having count(*) > 2;
```

Relational Model, SQL & PLSQL

Nested Sub Queries –

- Nested queries are a way to perform complex queries by embedding one query within another.
- The outer query can apply some conditions on the results of the inner query.
- There are mainly two types of nested queries:
 - Independent Nested Queries
 - Co-related Nested Queries
- **Independent Nested Queries -**
 - In this, query execution starts from innermost query to outermost queries.
 - The execution of inner query is independent of outer query, but the result of inner query is used in execution of outer query.
 - Various operators like IN, NOT IN, ANY, ALL, etc. are used in writing independent nested queries.

Relational Model, SQL & PLSQL

Nested Sub Queries –

➤ Co-related Nested Queries -

- In this, the output of inner query depends on the row which is being currently executed in outer query.
- A subquery is a select from where expression that is nested within another query.
- Mostly subqueries are used to perform tests for set membership to make set comparisons and determine set cardinality.
- **Set Membership** - SQL uses in and not in constructs for set membership tests.
 - **IN** - The in connective tests for set membership , where the set is a collection of values produced by a select clause.
 - Example - Get the details of author who have published book in year 2004.

```
select * from Author
where Author_name in (select Author_name
from Book
where Pub_year = '2004');
```


Relational Model, SQL & PLSQL

Nested Sub Queries –

➤ Set Membership –

- **NOT IN** - The not in connective tests for the absence of set membership.
- Example - Display title, author, and publisher name of all books except those which are published in year 2002, 2004 and 2005.

```
select Title, Author_name, Publisher_name, Pub_year  
from Book  
where Pub_year not in ('2002', '2004', '2005')
```

➤ Set Comparison –

- Nested subqueries are used to compare sets.
- SQL uses various comparison operators such as <, <=, >, >=, =, <>, any, all, and some, etc. to compare sets.

Relational Model, SQL & PLSQL

Modification of the Database –

➤ Different operations that modify the contents of the database are:

➤ **Delete**

➤ **Insert**

➤ **Update**

➤ **Delete –**

➤ A delete request is expressed in same way as a query.

➤ The delete operation is used to delete all or specific rows from database.

➤ We cannot delete values of particular attributes.

➤ Syntax –

```
delete from r  
where P;
```

where *r* is relation and *P* is predicate

Relational Model, SQL & PLSQL

Modification of the Database –

➤ Delete –

- The delete statement first finds all tuples in r for which $P(t)$ is true, and then deletes them from r .
- The where clause is omitted if all tuples in r are to be deleted.
- A delete command operates only on one relation.
- If we want to delete tuples from several relations, we must use one delete command for each relation.
- Example – Delete all books from Book relation where publishing year is less than 1997.

delete from Book

where pub_year < 1997;

Relational Model, SQL & PLSQL

Modification of the Database –

➤ Insertion –

- To insert data into a relation, we either specify a tuple to be inserted or write a query whose result is a set of tuples to be inserted.
- The attribute values for inserted tuples must be members of the attribute's domain.
- Example – consider customer relation (Cust_no, Cust_name, Cust_add, Cust_ph). Insert a new customer record with customer no. as 05, customer name as 'Pratik', Address- 'Kolhapur' and ph. no.as 1234567890.

insert into customer

values (05, 'Pratik', 'Kolhapur',1234567890);

- We can insert tuples in a relation on the basis of the result of a query.
- Consider two relations: Student = {Roll_No, Name, Flag} & Defaulter_student = {Roll_No, Name} .

Relational Model, SQL & PLSQL

Modification of the Database –

➤ Insertion –

- Suppose we want to insert tuples from Student relation to Defaulter_student where flag is 'D'.

```
insert into Defaulter_student
```

```
select Roll_no, Name
```

```
from Student
```

```
where flag = 'D',
```

Relational Model, SQL & PLSQL

Modification of the Database –

➤ Update –

- Using update statement, we can change value in a tuple or all the tuples of a relation.
- Consider relation: Employee = { Emp_code, Name, Salary}

Increase salary of all employees by 15%

update Employee

Set Salary = Salary* 1.15;